Object Oriented Programming (CSEG1044)

Unit 01 – Lecture 01: OOP Paradigm + Principles + Modelling

April 1, 2026

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Introduction to OOPs and Java Fundamentals

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Introduction to OOPs and Java Fundamentals
- **Course thread:** Use OOP Paradigm + Principles + Modelling to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Recall Before You Start

- **Programming basics**: Variables, loops, and functions from C still matter; OOP changes how they are organized. Main reference: Programming in C prerequisite.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Recall Before You Start

- **Programming basics**: Variables, loops, and functions from C still matter; OOP changes how they are organized. Main reference: Programming in C prerequisite.
- **Problem decomposition**: OOP still depends on breaking problems into clear responsibilities before coding. Main reference: Programming in C prerequisite.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Learning Outcomes

- Explain why OOP became popular for large software systems.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Learning Outcomes

- Explain why OOP became popular for large software systems.
- Define: abstraction, encapsulation, inheritance, polymorphism (with a small example).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Learning Outcomes

- Explain why OOP became popular for large software systems.
- Define: abstraction, encapsulation, inheritance, polymorphism (with a small example).
- Convert a short problem statement into a small class design (classes + responsibilities).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Terms to Know

- **Class**: A blueprint (type) describing data (fields) and operations (methods).
- **Object**: A runtime instance of a class (has identity + state + behavior).
- **State**: The data inside an object (values of its fields at a time).
- **Behavior**: What an object can do (methods).
- **Invariant**: A rule that must always remain true for an object (e.g., `balance >= 0`).
- **Interface (idea)**: A public view/contract of what operations are allowed (we will formalize this later in Java).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Why OOP? (vs procedural)* $\rightarrow$ *The 4 OOP principles (with intuition)* $\rightarrow$ *How to model a real-world entity*.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Why OOP? (vs procedural)* $\rightarrow$ *The 4 OOP principles (with intuition)* $\rightarrow$ *How to model a real-world entity*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Why OOP? (vs procedural)* $\rightarrow$ *The 4 OOP principles (with intuition)* $\rightarrow$ *How to model a real-world entity*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Unit 01: Introduction to OOPs

- Lecture focus: OOP Paradigm + Principles + Modelling

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Today: Roadmap

- OOP history and evolution; why OOP (vs procedural)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Today: Roadmap

- OOP history and evolution; why OOP (vs procedural)
- Principles: abstraction, encapsulation, inheritance, polymorphism

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Today: Roadmap

- OOP history and evolution; why OOP (vs procedural)
- Principles: abstraction, encapsulation, inheritance, polymorphism
- Modelling a real-world entity (identify classes/objects/responsibilities)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Why OOP? (motivation)

- Small programs: procedural style works well (few files, few developers).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Why OOP? (motivation)

- Small programs: procedural style works well (few files, few developers).
- Large programs: managing shared state becomes difficult.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Why OOP? (motivation)

- Small programs: procedural style works well (few files, few developers).
- Large programs: managing shared state becomes difficult.
- OOP idea: **data + operations together** as an **object**.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Why OOP? (motivation)

- Small programs: procedural style works well (few files, few developers).
- Large programs: managing shared state becomes difficult.
- OOP idea: **data + operations together** as an **object**.
- Benefit: better modularity, easier maintenance, clearer mapping to real-world entities.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

OOP building blocks

- **Class**: blueprint (fields + methods)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

OOP building blocks

- **Class**: blueprint (fields + methods)
- **Object**: instance at runtime (identity + state + behavior)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

OOP building blocks

- **Class**: blueprint (fields + methods)
- **Object**: instance at runtime (identity + state + behavior)
- **State**: values of fields

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

OOP building blocks

- **Class**: blueprint (fields + methods)
- **Object**: instance at runtime (identity + state + behavior)
- **State**: values of fields
- **Behavior**: methods that operate on the state

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

OOP building blocks

- **Class**: blueprint (fields + methods)
- **Object**: instance at runtime (identity + state + behavior)
- **State**: values of fields
- **Behavior**: methods that operate on the state
- **Invariant**: rule that must always hold (e.g., `balance >= 0`)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 1: Abstraction

- **What**: focus on *what* a thing does, not *how* it does it.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 1: Abstraction

- **What**: focus on *what* a thing does, not *how* it does it.
- **Why**: hides complexity; makes code easier to use and change.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 1: Abstraction

- **What**: focus on *what* a thing does, not *how* it does it.
- **Why**: hides complexity; makes code easier to use and change.
- **Example**: `BankAccount.deposit()` is visible; internal storage details are hidden.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 1: Abstraction

- **What**: focus on *what* a thing does, not *how* it does it.
- **Why**: hides complexity; makes code easier to use and change.
- **Example**: `BankAccount.deposit()` is visible; internal storage details are hidden.
- **Pitfall**: exposing internal details (public fields) breaks the model.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 2: Encapsulation

- **What**: keep data private; control changes via methods.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 2: Encapsulation

- **What**: keep data private; control changes via methods.
- **Why**: protects invariants; prevents accidental misuse.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 2: Encapsulation

- **What**: keep data private; control changes via methods.
- **Why**: protects invariants; prevents accidental misuse.
- **Example**: only `deposit/withdraw` can change `balance`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 2: Encapsulation

- **What**: keep data private; control changes via methods.
- **Why**: protects invariants; prevents accidental misuse.
- **Example**: only `deposit/withdraw` can change `balance`.
- **Pitfall**: setters that allow invalid states (e.g., `setBalance(-10)`).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 3: Inheritance

- **What**: models an **is-a** relationship (specialization).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 3: Inheritance

- **What**: models an **is-a** relationship (specialization).
- **Why**: reuse common behavior when relationship is truly is-a.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 3: Inheritance

- **What**: models an **is-a** relationship (specialization).
- **Why**: reuse common behavior when relationship is truly is-a.
- **Example**: `SavingAccount` is-a `BankAccount`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 3: Inheritance

- **What**: models an **is-a** relationship (specialization).
- **Why**: reuse common behavior when relationship is truly is-a.
- **Example**: `SavingAccount` is-a `BankAccount`.
- **Pitfall**: using inheritance only for code reuse; prefer composition for has-a.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 4: Polymorphism

- **What**: one interface/type, many implementations.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 4: Polymorphism

- **What**: one interface/type, many implementations.
- **Why**: enables flexible design (code works with parent types, not specific classes).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 4: Polymorphism

- **What**: one interface/type, many implementations.
- **Why**: enables flexible design (code works with parent types, not specific classes).
- **Example**: a `BankAccount` reference can point to different account subtypes.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Principle 4: Polymorphism

- **What**: one interface/type, many implementations.
- **Why**: enables flexible design (code works with parent types, not specific classes).
- **Example**: a `BankAccount` reference can point to different account subtypes.
- **Pitfall**: confusing **overloading** (compile-time) with **overriding** (runtime).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Modelling: from problem statement to classes

1. List 3–5 **scenarios** (what users do).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Modelling: from problem statement to classes

1. List 3–5 **scenarios** (what users do).
2. Extract **nouns** → candidate classes (Account, Customer).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Modelling: from problem statement to classes

1. List 3–5 **scenarios** (what users do).
2. Extract **nouns** → candidate classes (Account, Customer).
3. Extract **verbs** → candidate methods (deposit, withdraw).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Modelling: from problem statement to classes

1. List 3–5 **scenarios** (what users do).
2. Extract **nouns** → candidate classes (Account, Customer).
3. Extract **verbs** → candidate methods (deposit, withdraw).
4. Assign **responsibilities** (avoid one "God class").

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Modelling: from problem statement to classes

1. List 3–5 **scenarios** (what users do).
2. Extract **nouns** → candidate classes (Account, Customer).
3. Extract **verbs** → candidate methods (deposit, withdraw).
4. Assign **responsibilities** (avoid one "God class").
5. Write **invariants** and validate with sample cases.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *OOP Paradigm + Principles + Modelling* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Why OOP? (vs procedural)*, *The 4 OOP principles (with intuition)*, the worked example, or the recap?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Example: Model a real-world entity (1/2)

```
public class BankAccount {
    private final String accountNo;
    private double balance; // invariant: balance >= 0

    public BankAccount(String accountNo, double openingBalance)
        {
        if (openingBalance < 0) throw new
            IllegalArgumentException("openingBalance");
        this.accountNo = accountNo;
        this.balance = openingBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("
            amount");
        balance += amount;
    }
```

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., String, Scanner) (accessed April 1, 2026).

Example: Model a real-world entity (2/2)

```
// continued...
    public void withdraw(double amount) {
        if (amount <= 0 || amount > balance)
            throw new IllegalArgumentException("amount");
        balance -= amount;
    }
}
```

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Model `Student`: 3 fields + 3 methods + 1 invariant.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Model `Student`: 3 fields + 3 methods + 1 invariant.
2. Decide: is-a or has-a? Car–Engine, Dog–Animal.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Model `Student`: 3 fields + 3 methods + 1 invariant.
2. Decide: is-a or has-a? Car–Engine, Dog–Animal.
3. Add `transferTo(other, amount)` to `BankAccount`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Watch-outs

- Treating a class as only a data bucket with no behavior.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Watch-outs

- Treating a class as only a data bucket with no behavior.
- Confusing one object instance with the class blueprint.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Watch-outs

- Treating a class as only a data bucket with no behavior.
- Confusing one object instance with the class blueprint.
- Using inheritance before the responsibilities are understood.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- OOP helps manage complexity by organizing code around objects (state + behavior).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- OOP helps manage complexity by organizing code around objects (state + behavior).
- Abstraction + encapsulation simplify and protect designs.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- OOP helps manage complexity by organizing code around objects (state + behavior).
- Abstraction + encapsulation simplify and protect designs.
- Inheritance + polymorphism enable reuse and flexibility (use carefully).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- OOP helps manage complexity by organizing code around objects (state + behavior).
- Abstraction + encapsulation simplify and protect designs.
- Inheritance + polymorphism enable reuse and flexibility (use carefully).
- Modelling is the bridge from real-world requirements to classes and responsibilities.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint Questions

1. Why do we keep fields private in OOP?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint Questions

1. Why do we keep fields private in OOP?
2. Give one example each: abstraction and encapsulation.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint Questions

1. Why do we keep fields private in OOP?
2. Give one example each: abstraction and encapsulation.
3. Give one example each: is-a and has-a.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Lecture Preview

Next: Java Overview + Program Structure + Features

- Install/verify JDK; practice compile/run on your machine.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Lecture Preview

Next: Java Overview + Program Structure + Features

- Install/verify JDK; practice compile/run on your machine.
- Bring one question about JVM/JDK/JRE (confusing terms are expected!).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).